

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт перспективной инженерии

Дисциплина «Введение в программирование и алгоритмы»

Отчет по лабораторной работе №2
«Переменные и арифметические операции»

Выполнил студент группы
ПИЖ-б-о-26-1(2)
Киркачев М.Е.
Проверил:
Преподаватель по дисциплине
Николаев Е. И.

г. Ставрополь, 2026 г.

Цель работы: Научиться объявлять переменные базовых типов (int, double, float, char, bool), выполнять арифметические операции (+, -, *, /, %), правильно выбирать тип данных в зависимости от решаемой задачи, понимать механизмы явного и неявного приведения типов, избегать потери точности при вычислениях, корректно работать с целочисленным и вещественным делением.

ХОД РАБОТЫ

https://github.com/Max2311008/my-projects/tree/main/work%20in%20C%2B%2B/lab_2

Вариант №9

Задание 1 (базовый уровень)

Код

```
#include <iostream> //Подключаем заголовочный файл
#include <Windows.h>
int main() { //Объявляем главную функцию
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
    // 2, 4, 5 строки обрабатывают кириллицу
    int kilometers, meters; //Создаём переменные
    std::cout << "Перевод км в м" << std::endl;
    std::cout << "Количество киллометров = ";
    //Выводим текст на экран
    std::cin >> kilometers; //Вводим переменную с клавиатуры
    if (std::cin.fail() || kilometers < 0) {
        std::cerr << "Ошибка, количество киллометров должно быть число больше нуля";
        // Условие при котором не может быть отрицательных числе при таких величинах
    }
    else {
        meters = kilometers * 1000; //Вычисляем количество метров
        std::cout << "Количество метров = ";
        std::cout << meters; //Выводим результат на экран
    }
    return 0;
}
```

Результат

```
Перевод км в м  
Количество киллометров = 54  
Количество метров = 54000
```

Результат при вводе неверных данных

```
Перевод км в м  
Количество киллометров = -6  
Ошибка, количество киллометров должно быть число больше нуля
```

```
Перевод км в м  
Количество киллометров = а  
Ошибка, количество киллометров должно быть число больше нуля
```

Задание 2 (средний уровень)

Код

```
#include <iostream> //Подключаем заголовочный файл
#include <Windows.h>
#include <iomanip> // библиотека для fixed и setprecision()
int main() { //Объявляем главную функцию
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
    // 2, 5, 6 строки обрабатывают кириллицу
    int height; // Создание переменной
    double length, width; // Создание переменных с точкой, так как эти переменные могут иметь дробное значение
    double volume, squarefs; // Создание переменных с точкой, так как эти переменные могут иметь дробное значение
    std::cout << "Вычисление объёма параллелепипеда и площади поверхности" << std::endl; // Вывод текста на экран
    std::cout << "ДЛИНА ПАРАЛЛЕЛЕПИПЕДА = "; // Вывод текста на экран
    std::cin >> length; // Введение данных переменной с клавиатуры
    std::cout << "Число после округления = " << std::fixed << std::setprecision(2) << length << std::endl;
    // Округление числа до сотых
    std::cout << "ШИРИНА ПАРАЛЛЕЛЕПИПЕДА = "; // Вывод текста на экран
    std::cin >> width; // Введение данных переменной с клавиатуры
    std::cout << "Число после округления = " << std::fixed << std::setprecision(2) << width << std::endl;
    // Округление числа до сотых
    std::cout << "ВЫСОТА ПАРАЛЛЕЛЕПИПЕДА = "; // Вывод текста на экран
    std::cin >> height; // Введение данных переменной с клавиатуры
    if (std::cin.fail() || length < 0 || width < 0 || height < 0) {
        std::cerr << "ОШИБКА, числа в этих переменных должны быть положительными";
    } // Условие что переменные должны быть числами больше нуля
    else { // Иначе если условия не выполняются, то выводим продолжение кода
        volume = static_cast<double>(length * width * height); // явное приведение операндов к double
        squarefs = static_cast<double>(2 * (length * width + width * height + length * height));
        // явное приведение операндов к double
        std::cout << "ОБЪЁМ ПАРАЛЛЕЛЕПИПЕДА = " << volume << std::endl;
        std::cout << "Результат выражения после округления = " << std::fixed << std::setprecision(2) << volume << std::endl;
        // Округление числа до сотых
        std::cout << "ПЛОЩАДЬ ПОЛНОЙ ПОВЕРХНОСТИ ПАРАЛЛЕЛЕПИПЕДА = " << squarefs << std::endl;
        std::cout << "Результат выражения после округления = " << std::fixed << std::setprecision(2) << squarefs << std::endl;
        // Округление числа до сотых
    }

    return 0; // Вывод результата
}
```

Результат

```
Вычисление объёма параллелепипеда и площади поверхности
ДЛИНА ПАРАЛЛЕЛЕПИПЕДА = 6.534
Число после округления = 6.53
ШИРИНА ПАРАЛЛЕЛЕПИПЕДА = 7.21442
Число после округления = 7.21
ВЫСОТА ПАРАЛЛЕЛЕПИПЕДА = 64
ОБЪЁМ ПАРАЛЛЕЛЕПИПЕДА = 3016.90
Результат выражения после округления = 3016.90
ПЛОЩАДЬ ПОЛНОЙ ПОВЕРХНОСТИ ПАРАЛЛЕЛЕПИПЕДА = 1854.08
Результат выражения после округления = 1854.08
```

Результат ввода неверных переменных

```
Вычисление объёма параллелепипеда и площади поверхности
ДЛИНА ПАРАЛЛЕЛЕПИПЕДА = -5
Число после округления = -5.00
ШИРИНА ПАРАЛЛЕЛЕПИПЕДА = -32
Число после округления = -32.00
ВЫСОТА ПАРАЛЛЕЛЕПИПЕДА = -23
ОШИБКА, числа в этих переменных должны быть положительными
```

Задание 3(повышенный уровень)

Код

```
#define NOMINMAX // Запретит Windows.h определять макросы min и max
#include <iostream> //Подключаем заголовочный файл
#include <limits>
#include <Windows.h>
#include <iomanip> // библиотека для fixed и setprecision()
#include <cmath>
int main() { //Объявляем главную функцию
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
    // 4, 8, 9 строки обрабатывают кириллицу
    int number1, number2; //Создание переменных с целыми числами
    double number3, number4; //Создание переменных с числами с точкой
    while (true) { //Ввод числе в int
        std::cout << "Введите number1: ";
        if (std::cin >> number1) {
            std::cout << "Введите number2: ";
            if (std::cin >> number2)
                break;
        } //Условие повторного ввода данных при ошибке

        std::cout << "Ошибка! Введите число заново.\n"; //Вывод текста на экран

        std::cin.clear(); // сброс флага ошибки
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // очистка буфера от мусора
    }
    while (true) { //Ввод числе в double
        std::cout << "Введите number3 ";
        if (std::cin >> number3) {
            std::cout << "Введите number4: ";
            if (std::cin >> number4)
                break;
        } //Условие повторного ввода данных при ошибке

        std::cout << "Ошибка! Введите число заново.\n"; //Вывод текста на экран

        std::cin.clear(); // сброс флага ошибки
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // очистка буфера от мусора
    }
    std::cout << "\n$=== int ===$\n";
    std::cout << "Сумма = " << number1 + number2 << '\n'; // Вывод суммы(int) на экран
    std::cout << "Разность = " << number1 - number2 << '\n'; // Вывод разности(int) на экран
```

```

std::cout << "Произведение = " << number1 * number2 << '\n'; // Вывод произведения(int) на экран
if (number2 != 0) {
    std::cout << "Частное = " << number1 / number2 << '\n'; // Вывод частного(int) на экран
    std::cout << "Остаток от деления = " << number1 % number2; // Вывод остатка(int) на экран
}
else {
    std::cerr << "Нельзя делить на 0!!!!";
} // Вывод на экран что на ноль делить нельзя

std::cout << "\n$=== double ===$\n";
std::cout << "Сумма = " << number3 + number4 << '\n'; // Вывод суммы(double) на экран
std::cout << "Разность = " << number3 - number4 << '\n'; // Вывод разности(double) на экран
std::cout << "Произведение = " << number3 * number4 << '\n'; // Вывод произведения(double) на экран
if (std::fabs(number4) != 0.0) {
    std::cout << "Частное = " << number3 / number4 << '\n'; // Вывод частного(double) на экран
}
else {
    std::cerr << "Нельзя делить на 0!!!!";
} // Вывод на экран что на ноль делить нельзя

std::cout << "\n$=== явное приведение ===$\n";
if (number2 != 0) { // Условие что number2 не равно 0
    std::cout << "Деление без явного приведения = " << number1 / number2 << '\n';
    std::cout << "Деление с явным приведением = " << static_cast<double>(number1) / number2;
    // явное приведение к операнду double
}

std::cout << "\n$=== усечение vs округление ===$\n";
double number5 = 5.6;
std::cout << "Усечение = " << std::trunc(number5) << '\n'; // Изменение числа при усечении
std::cout << "Округление = " << std::round(number5) << '\n'; // Изменение числа при округлении

std::cout << "\n$=== float vs double ===$\n";
float a = 1.0f / 3.0f;
double b = 1.0 / 3.0;
std::cout << std::fixed << std::setprecision(14);
std::cout << "float: " << a << '\n';
std::cout << "double: " << b << '\n';

std::cout << "\n$===== ТАБЛИЦА СРАВНЕНИЯ int vs double =====$\n";
std::cout << std::left << std::setw(16) << "Операция" //16 символов слева
    << std::right << std::setw(14) << "int" // 14 символов справа
    << std::setw(16) << "double" << '\n'; //16 символов справа
std::cout << std::string(46, '-') << '\n'; // строка шириной 46 -

std::cout << std::fixed << std::setprecision(4);

std::cout << std::left << std::setw(16) << "Сумма" //16 символов слева
    << std::right << std::setw(14) << (number1 + number2) // 14 символов справа
    << std::setw(16) << (number3 + number4) << '\n'; //16 символов справа

std::cout << std::left << std::setw(16) << "Разность" //16 символов слева
    << std::right << std::setw(14) << (number1 - number2) //14 символов справа
    << std::setw(16) << (number3 - number4) << '\n'; //16 символов справа

std::cout << std::left << std::setw(16) << "Произведение" //16 символов слева
    << std::right << std::setw(14) << (number1 * number2) //14 символов справа
    << std::setw(16) << (number3 * number4) << '\n'; //16 символов справа

if (number2 != 0 && std::fabs(number4) != 0.0) { //Условие что number2 и модуль number4 не равны 0
    std::cout << std::left << std::setw(16) << "Частное" //16 символов слева
        << std::right << std::setw(14) << (number1 / number2) //14 символов справа
        << std::setw(16) << (number3 / number4) << '\n'; //16 символов справа
    }
if (number2 != 0) {
    std::cout << std::left << std::setw(16) << "Остаток" //16 символов слева
        << std::right << std::setw(14) << (number1 % number2) //14 символов справа
        << std::setw(16) << "н/д" << '\n'; //16 символов справа
    }

return 0; //Вывод результат
}

```

Результат

```

Введите number1: 65
Введите number2: 36
Введите number3 8.94
Введите number4: 342.2

$=== int ===$
Сумма = 101
Разность = 29
Произведение = 2340
Частное = 1
Остаток от деления = 29
$=== double ===$
Сумма = 351.14
Разность = -333.26
Произведение = 3059.27
Частное = 0.0261251

$=== явное приведение ===$
Деление без явного приведения = 1
Деление с явным приведением = 1.80556
$=== усечение vs округление ===$
Усечение = 5
Округление = 6

$=== float vs double ===$
float: 0.333333334326744
double: 0.333333333333333

$===== ТАБЛИЦА СРАВНЕНИЯ int vs double =====$
Операция          int          double
-----
Сумма              101          351.1400
Разность           29          -333.2600
Произведение       2340         3059.2680
Частное            1            0.0261
Остаток            29           н/д

```

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Приведение типов – преобразование значения одного типа в значение другого типа.
2. Неявное — компилятор выполняет автоматически, без указания программиста.

Явное — программист специально указывает оператор приведения.

3. Ситуации:

- При присваивании
 - В арифметических выражениях
 - При передаче аргументов в функции
4. Так как действия проходят над обоими операндами типа int, то выполняется целочисленное деление 5/2. Затем результат неявно приводится к double = 2.0
 5. double result2 = static_cast<double>(a) / b;
 6. static_cast – современное приведение в C++, проверяется на этапе компиляции. Лучше C-style(int), т.к. обеспечивает контроль преобразования

7. Произойдёт усечение – отброс дробной части (все числа после точки)
8. Для выполнения действия(%) требуются целые числа:
`int r = static_cast<int>(x) % static_cast<int>(y)`
9. truncation (усечение) – отбрасывает дробную часть.
rounding (округление) – математическое округление числа в большую сторону.
10. Потеря точности (precision loss) — это ситуация, когда при приведении вещественного значения к типу с меньшей разрядностью часть значащих цифр после запятой теряется, и результат становится неточным.

```
float a = 1.0f / 3.0f;  
double b = 1.0 / 3.0;  
std::cout << std::fixed << std::setprecision(14);  
std::cout << "float: " << a << '\n';  
std::cout << "double: " << b << '\n';
```

где float ответ будет 0.33333334326744

где double ответ будет 0.33333333333333

11. Выход за диапазон типа. `int max + 1` станет `-2_147_483_648`
12. Переменная — это именованная область памяти, имеющая тип, который задаёт её размер, диапазон значений и допустимые операции.
Тип данных – характеристика переменной, которая определяет размер памяти, диапазон значений и набор допустимых операций.
13. Переменная — это именованная область памяти, имеющая тип, который задаёт её размер, диапазон значений и допустимые операции.
- Компиляция – перевод кода в файл, проверяя синтаксис и типы.
 - Запуск – загрузка программы в память ОС и начала исполнения
 - Выполнение – работа программы (ввод, вывод) вычисления и тд
14. Имя должно быть осмысленным (отражать смысл значения); не использовать однобуквенные имена без пояснения; имя может содержать буквы, цифры, знак `_`; нельзя использовать пробелы и спецсимволы; имя не может начинаться с цифры.
- Примеры правильных имён: `length` - длина, `students` – количество студентов.
 - Примеры неправильных имён: `a,b,c` – не осмысленные имена;
`1length` – начинается с цифры

15.

Тип	Размер (байт)	Диапазон	Когда использовать
int	4 байта	$-2^{31} \dots 2^{31} - 1$	целые числа
double	8 байт	$\pm 1.7e\pm 308$	числа с плавающей точкой(вещественные)
float	4 байта	$\pm 3.4e\pm 38$	числа с плавающей точкой(вещественные) для экономии памяти
char	1 байт	-128 ... 127	символы
bool	1 байт	true / false	логика

16. int(есть и отрицательные и положительные числа), unsigned int(только положительные)

17. Студенты – int(их целое число), цена – double(может быть дробной), рост – double(может быть дробный), Транзакция – double(может быть дробной).

18. Неявное преобразование – преобразование, которое компилятор выполняет автоматически, без указания программиста.

Явное преобразование – преобразование, которое программист указывает в коде с помощью различных операторов.

19.

```
int a = 3.14;  
double b = 5;  
char c = 'A'; int d = c;
```

20. static_cast — оператор явного приведения, проверяемый на этапе компиляции. Лучше C-style тем, что безопаснее, он явно показывает намерение, легко ищется в коде

21. Потому что оба числа имеют тип данных int и результат приводится к double(2.0).

22. 5.0 / 2

static_cast<double>(5) / 2

double(5) / 2

23.

```
int a = 3.14; // 3
double b = 5; //5.0
char c = 65; // A
int d = c; // 65
```

24. В первом случае отбрасывается дробная часть, во втором происходит действие без потерь, в третьем выводит A, так как это код для символа A, а в четвертом 65, так как символ A перевели в int.

25. Потеря точности (precision loss) — это ситуация, когда при приведении вещественного значения к типу с меньшей разрядностью часть значащих цифр после запятой теряется, и результат становится неточным.

```
float a = 1.0f / 3.0f;
double b = 1.0 / 3.0;
std::cout << std::fixed << std::setprecision(14);
std::cout << "float: " << a << '\n';
std::cout << "double: " << b << '\n';
```

где float ответ будет 0.33333334326744

где double ответ будет 0.33333333333333

26. truncation (усечение) – отбрасывает дробную часть.

rounding (округление) – математическое округление числа в большую сторону.

27. Выход за диапазон типа. int max + 1 станет -2_147_483_648

28. Оператор % только для целых чисел. Остаток для вещественных происходит через std::fmod().

29. () – 1 приоритет (слева направо)

*, /, % - 3 приоритет (слева направо)

+, - - 4 приоритет (слева направо)

30. Если оба операнда целые, то / выполняет целочисленное деление (отбрасывает дробную часть), а если хотя бы один имеет тип вещественного числа, то дробная часть сохранится.

31. Ошибка компиляции, должна стоять запятая, а не точка.

32.

```
double a, b, c;
double avg = (a + b + c) / 3.0;
```

33. Числа без пояснения в коде, заменяются const для читаемости и удобства изменения.

34. std::cout << «...» - вывод текста на экран

std::cin >> «...» - ввод текста с клавиатуры

35. std::endl – переводит строку на новую и сбрасывает буфер

\n – только переводит строку на новую

36.

```
#include <iomanip>
#include <iostream>
int main() {
    double x = 3.4533;
    std::cout << std::fixed << std::setprecision(3) << x;
    return 0;
}
```

37. Std::fixed – фиксирует формат с плавающей точкой
 Std::setprecision() – задаёт количество знаков после запятой
 38. Std:: cout << a << «...» << b << «...» << c << std::endl;

39.

```
#include <iostream>
#include <Windows.h>
int main() {
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
    int a, b;
    std::cout << "Введите числа:";
    std::cin >> a >> b;
    double c = static_cast<double>(a) / b;
    std::cout << "Результат:" << c;
    return 0;
}
```

40. Не равны
 41. $Fabs(a-b) < 1e-9$
 42. 3.14 → double; 3.14f → float; 42L → long; 42U → unsigned int
 43. Const – значение, которое нельзя изменить, это нужно для защиты от случайного изменения и читаемости.
 44. Команда, которая помогает ловить несоответствие типов до запуска
 45. Преобразование значения одного типа в другой

Вывод: Благодаря этой лабораторной работе я научился объявлять переменные базовых типов (int, double, float, char, bool), выполнять арифметические операции (+, -, *, /, %), правильно выбирать тип данных в зависимости от решаемой задачи, понимать механизмы явного и неявного приведения типов, избегать потери точности при вычислениях, корректно работать с целочисленным и вещественным делением.